Session 1: Data and Coordinates

Units, sky coordinates, time, distances, Gaia

astro-lab – reading notes

Goals

After this session you can (i) carry units through a calculation without losing track of dimensions, (ii) convert a sky position between ICRS, Galactic and horizontal coordinates, (iii) explain why astronomy uses several time scales, and (iv) turn a Gaia parallax and magnitude into an absolute magnitude for an HR diagram.

1 Units and dimensional analysis

Astronomical quantities span tens of orders of magnitude, so errors usually come from unit mistakes rather than algebra. `astropy.units` attaches a unit to every number and refuses to add incompatible ones.

Worked example (Kepler's third law). For a small body orbiting a mass M on a circular orbit of radius a, gravity supplies the centripetal force:

$$\frac{GMm}{a^2} = m\,\omega^2 a \quad \Rightarrow \quad P = \frac{2\pi}{\omega} = 2\pi\sqrt{\frac{a^3}{GM}}.$$

For $a = 1\,\mathrm{au}$ and $M = M_\odot$ this gives $P \approx 365.26$ d, which is what the script prints. The same formula holds for ellipses with a the semi-major axis.

2 Sky coordinates

- **ICRS** (RA, Dec): the modern equatorial frame, fixed to distant quasars. Catalogs such as Gaia use it.
- **Galactic** (ℓ, b): longitude measured from the Galactic centre, latitude from the Galactic plane. Useful for stellar populations.
- **Horizontal** (Alt, Az): depends on observer location and time; what a telescope actually points to.

Converting between frames is a rotation of the unit vector on the sphere. In Astropy it is one call, `coord.transform_to(frame)`, but you should know the rotation exists: write RA/Dec as $\hat{n} = (\cos\delta\cos\alpha,\ \cos\delta\sin\alpha,\ \sin\delta)$ and apply a fixed 3×3 rotation matrix for ICRS→Galactic.

3 Time

Several time scales exist because Earth's rotation is irregular:

Scale	Use
UTC	civil time, has leap seconds
TAI	continuous atomic time
TT	time for geocentric ephemerides (TT = TAI + 32.184 s)
TDB	barycentric time, used for solar-system and pulsar timing

Julian Date (JD) counts days continuously and is convenient for light curves. Always record the scale together with the number.

4 Distances and the HR diagram

Parallax ϖ (in arcseconds) gives $d = 1/\varpi$ parsecs. With ϖ in milliarcseconds, the absolute magnitude from an apparent magnitude m is

$$M = m + 5\log_{10}\!\left(\frac{\varpi}{100\,\mathrm{mas}}\right).$$

Derivation: $M = m - 5\log_{10}(d/10\,\mathrm{pc})$ and $d = 1000/\varpi_{\mathrm{mas}}\,\mathrm{pc}$.

Plot M_G (inverted axis) against the colour $G_{BP} - G_{RP}$ for stars with `parallax_over_error` > 10. You should see the main sequence, the red giant branch and the white dwarf sequence. Selecting on parallax precision introduces a bias; note it in your write-up.

5 FITS in one paragraph

A FITS file is a sequence of header–data units (HDUs). Each header is ASCII `KEY = value` cards; the data is an image or table. Read with `astropy.io.fits.open`, and always inspect `hdul.info()` before assuming where the data lives.

6 Code walkthrough

The repository script `astropy_basics.py` covers units, a coordinate conversion, an altitude calculation and time scales. The Gaia query is commented out because it needs network access.

```python
"""Session 1: units, coordinates, time. Offline; Gaia query is at the bottom (needs network)."""
import astropy.units as u
from astropy.constants import G, M_sun, au
from astropy.coordinates import SkyCoord, EarthLocation, AltAz
from astropy.time import Time

# Units: orbital period of Earth from Kepler's third law
a = 1 * u.au
P = (2 * 3.141592653589793 * (a**3 / (G * M_sun)) ** 0.5).to(u.day)
print(f"Earth period: {P:.2f}")  # ~365.25 d

# Coordinates: Sirius in ICRS -> Galactic
sirius = SkyCoord("06h45m08.9s", "-16d42m58s", frame="icrs")
print("Galactic l,b:", sirius.galactic.l.deg, sirius.galactic.b.deg)

# Altitude from Bangkok right now
bkk = EarthLocation(lat=13.75 * u.deg, lon=100.5 * u.deg, height=2 * u.m)
now = Time.now()
altaz = sirius.transform_to(AltAz(obstime=now, location=bkk))
```

```
print(f"Sirius altitude from Bangkok at {now.iso}: {altaz.alt:.1f}")

# Time scales
t = Time("2026-10-25 00:00:00", scale="utc")
print("JD:", t.jd, "| TDB:", t.tdb.iso)

# Gaia query (uncomment with network access):
# from astroquery.gaia import Gaia
# job = Gaia.launch_job("SELECT TOP 1000 source_id, ra, dec, parallax,
    phot_g_mean_mag, bp_rp "
#                       "FROM gaiadr3.gaia_source WHERE parallax_over_error >
    10")
# df = job.get_results().to_pandas()
# -> plot bp_rp vs absolute mag (G + 5*log10(parallax/100)) for the HR diagram
```

7 Exercises

1. Compute Mars' orbital period from $a = 1.524$ au and compare with 687 d.
2. Convert the coordinates of three bright stars from ICRS to Galactic and explain which one is closest to the Galactic plane.
3. Explain the number 69.18 s: the script shows TDB is about 69.18 s ahead of UTC on 2026-10-25. Break it into the 32.184 s TT−TAI offset, the accumulated leap seconds (TAI−UTC), and the small periodic TDB−TT term (at most about 1.7 ms).
4. Query 10 000 Gaia stars and make the HR diagram. Repeat with `parallax_over_error` > 5 and describe what changes.

References

- Astropy tutorials: https://learn.astropy.org
- Astropy documentation for `units`, `coordinates`, `time`
- Gaia Collaboration, *Gaia Data Release 3* (documentation on the ESA Gaia site)